

REQUI TRADING

Predetermined Market Intelligence + LLM Orchestration Revision

OBJECTIVE

The next revision should make Requi capable of answering general, beginner-level market questions using predetermined backend workflows.

The LLM should NOT determine market conditions from its internal knowledge.

The LLM should NOT require the user to know a ticker.

The LLM should NOT ask unnecessary questions such as which country or exchange the user means when the request is clearly about the stock market.

Instead, the application should classify the user's intent and automatically execute the correct market intelligence functions before the LLM generates its response.

CORE ARCHITECTURE:

USER QUESTION

- INTENT CLASSIFICATION
- PREDETERMINED TOOL ROUTING
- LIVE DATA RETRIEVAL
- DETERMINISTIC CALCULATIONS
- MARKET CLASSIFICATION

- STRUCTURED JSON
- LLM INTERPRETATION
- UI COMPONENTS
- USER RESPONSE

1. DEFAULT MARKET CONFIGURATION

For the current Requi release, all unspecified stock-market questions should default to the United States.

Application constants:

DEFAULT_MARKET = “US”

DEFAULT_EXCHANGES = [
“NYSE”,
“NASDAQ”
]

DEFAULT_CURRENCY = “USD”

MARKET_TIMEZONE = “America/New_York”

Do not ask the user:

“What country?”

“Which market?”

“Which exchange?”

unless the user specifically requests an international market.

Examples:

“How’s the market today?”

→ US market

“What stocks are moving?”

→ NYSE/NASDAQ

“What should I buy today?”

→ US equities

“How is Japan doing?”

→ override default and route to Japan if/when international support exists.

2. CREATE A MARKET INTENT ROUTER

Before the main LLM generates an answer, classify every financial question into a predetermined intent.

Example intents:

GENERAL_MARKET
STOCK_DISCOVERY
TICKER_ANALYSIS

MARKET_MOVERS
SECTOR_ANALYSIS
INDUSTRY_ANALYSIS
EARNINGS_CALEDNAR
ECONOMIC_CALEDNAR
TECHNICAL_ANALYSIS
NEWS_ANALYSIS
SENTIMENT_ANALYSIS
PORTFOLIO_ANALYSIS
TRADE_REQUEST
GENERAL_EDUCATION

Example TypeScript:

```
type MarketIntent =  
| "GENERAL_MARKET"  
| "STOCK_DISCOVERY"  
| "TICKER_ANALYSIS"  
| "MARKET_MOVERS"  
| "SECTOR_ANALYSIS"  
| "INDUSTRY_ANALYSIS"  
| "EARNINGS_CALEDNAR"  
| "ECONOMIC_CALEDNAR"  
| "TECHNICAL_ANALYSIS"  
| "NEWS_ANALYSIS"  
| "SENTIMENT_ANALYSIS"  
| "PORTFOLIO_ANALYSIS"  
| "TRADE_REQUEST"  
| "GENERAL_EDUCATION";
```

3. BEGINNER / DUMMY-PROOF QUERY MAPPING

Users should not need to understand financial terminology.

Map natural-language questions to predetermined workflows.

Example:

```
const intentPatterns = {  
  
  GENERAL_MARKET: [  
    "how is the market",  
    "how's the market",  
    "what is the market doing",  
    "is the market good today",  
    "is today a good trading day",  
    "what's happening in the market"  
  ],  
  
  STOCK_DISCOVERY: [  
    "what should i buy",  
    "what stock should i buy",  
    "give me stocks to watch",  
    "what looks good",  
    "where should i put my money",  
  ],  
}
```

“find me a stock”

],

MARKET_MOVERS: [

“what’s moving”,

“what’s hot”,

“what stocks are running”,

“biggest movers”,

“what’s crashing”

],

EARNINGS_CALENDAR: [

“who reports today”,

“earnings today”,

“earnings tomorrow”,

“quarterly earnings”,

“who reports after close”,

“who reports before open”

]

};

Semantic classification should supplement these patterns so the system is not dependent on exact wording.

4. GENERAL MARKET ANALYSIS PIPELINE

Questions such as:

“How’s the market today?”

must automatically trigger:

runGeneralMarketAnalysis()

This function should retrieve and calculate:

1. Major index performance
2. Market breadth
3. Sector performance
4. Volatility
5. Market momentum
6. Trading volume/participation
7. Macro events
8. Important market news
9. Market regime
10. Overall Market Health Score

The LLM should receive the completed analysis.

It should NOT independently guess these values.

5. MAJOR INDEX ANALYSIS

Track at minimum:

SPY

QQQ

DIA

IWM

For each:

current_price
previous_close
daily_return
5_day_return
20_day_return
SMA20
SMA50
SMA200
price_vs_SMA20
price_vs_SMA50
price_vs_SMA200
volume
average_volume
relative_volume
timestamp
source

6. MARKET BREADTH ENGINE

Calculate:

advancing_stocks
declining_stocks
unchanged_stocks
advance_decline_ratio
percent_advancing

percent_above_SMA20

percent_above_SMA50

percent_above_SMA200

new_52_week_highs

new_52_week_lows

Example:

$$\text{percentAdvancing} = \frac{\text{advancingStocks}}{(\text{advancingStocks} + \text{decliningStocks})} * 100$$

Suggested classification:

= 65 = STRONG

55–64.99 = POSITIVE

45–54.99 = MIXED

35–44.99 = WEAK

< 35 = VERY_WEAK

7. SECTOR LEADERSHIP ENGINE

Track major U.S. sectors.

For each sector calculate:

daily_return
5_day_return
20_day_return
relative_strength_vs_SPY
volume_ratio
percent_components_advancing

Generate:

sector_score

Then classify:

LEADING
STRONG
NEUTRAL
WEAK
LAGGING

Return ranked sectors.

Example:

```
[  
{  
  "sector": "Technology",  
  "daily_return": 1.4,  
  "relative_strength": 0.8,  
  "classification": "LEADING"  
}  
]
```

8. VOLATILITY ENGINE

Use:

VIX

VIX daily change

VIX 5-day change

SPY realized volatility

QQQ realized volatility

intraday index range

Basic VIX classification:

$VIX < 15$

LOW_VOLATILITY

15–20

NORMAL

20–25

ELEVATED

25–30

HIGH

30

EXTREME

Do not use VIX by itself to determine the entire market regime.

9. MARKET HEALTH SCORE

Create a deterministic 0–100
MARKET_HEALTH_SCORE.

Initial formula:

MARKET_HEALTH_SCORE =

(IndexTrend $\times$ 0.25)

+

(MarketBreadth $\times$ 0.20)

+

(SectorParticipation $\times$ 0.15)

+

(VolatilityScore $\times$ 0.15)

+

(MomentumScore $\times$ 0.10)

+

(VolumeParticipation $\times$ 0.10)

+

(MacroRiskScore $\times$ 0.05)

All components must first be normalized to 0–100.

Example implementation:

```
function calculateMarketHealth(input) {
```

```
  return (
```

```
    input.indexTrend * 0.25 +
```

```
        input.marketBreadth * 0.20 +
        input.sectorParticipation *
0.15 +
        input.volatilityScore * 0.15 +
        input.momentumScore * 0.10 +
        input.volumeParticipation *
0.10 +
        input.macroRiskScore * 0.05
    );
}
```

Round result to nearest integer.

10. MARKET REGIME ENGINE

The system should classify current market conditions before the LLM receives the information.

Possible states:

STRONG_BULL

BULL

CAUTIOUS_BULL

MIXED

SIDEWAYS

CAUTIOUS_BEAR

BEAR

HIGH_VOLATILITY

Example initial logic:

```
if (  
    marketHealth >= 75 &&  
    breadth >= 60 &&  
    indexTrend >= 70  
) {  
    regime = "STRONG_BULL";  
}  
  
else if (  
    marketHealth >= 60 &&  
    breadth >= 50  
) {  
    regime = "BULL";  
}  
  
else if (  
    indexTrend >= 60 &&  
    breadth < 50  
) {  
    regime = "CAUTIOUS_BULL";  
}  
  
else if (  
    marketHealth >= 45 &&  
    marketHealth < 60  
) {  
    regime = "MIXED";  
}
```

```
else if (  
volatilityScore <= 25  
) {  
regime = "HIGH_VOLATILITY";  
}
```

```
else if (  
marketHealth < 35  
) {  
regime = "BEAR";  
}
```

These thresholds should be treated as V1 parameters and later calibrated/backtested rather than permanently hard-coded.

11. STOCK DISCOVERY ENGINE

The question:

“What stock should I buy today?”

must NOT immediately be sent to the LLM for ticker generation.

Instead execute:

runStockDiscovery()

Pipeline:

MARKET REGIME

- SECTOR RANKING
- INDUSTRY RANKING
- LIQUID STOCK UNIVERSE
- RELATIVE STRENGTH
- MOMENTUM
- VOLUME
- CATALYST
- NEWS
- FUNDAMENTAL FILTER
- RISK FILTER
- RANKED CANDIDATES

The LLM should only discuss stocks returned by this pipeline.

12. INITIAL STOCK RANKING MODEL

Example:

STOCK_SCORE =

20% Relative Strength

+

15% Sector Strength

+

15% Industry Strength

+

15% Momentum
+
10% Relative Volume
+
10% Catalyst
+
10% Fundamental Quality
+
5% Sentiment
Return 0–100.

Example output:

```
{  
  "ticker": "XYZ",  
  "score": 84,  
  "sector_rank": 2,  
  "relative_volume": 1.8,  
  "relative_strength": 91,  
  "catalyst": "...",  
  "risk_level": "MODERATE"  
}
```

The LLM explains WHY the candidate ranked highly.

It should not invent the candidate.

13. DATA GATEWAY

Create one normalized market-data gateway.

Example:

`MarketDataGateway.getQuote(symbol)`

`MarketDataGateway.getHistory(symbol)`

`MarketDataGateway.getMarketBreadth()`

`MarketDataGateway.getSectorPerformance()`

`MarketDataGateway.getNews()`

`MarketDataGateway.getEarnings()`

`MarketDataGateway.getEconomicCalendar()`

`MarketDataGateway.getIndicators()`

14. DATA PROVIDER FALLBACK

Do not return:

“I don’t have verified live market data”

after one provider fails.

Implement provider fallback.

Example:

```
async function getVerifiedQuote(symbol) {
```

```
  const providers = [
```

```
    brokerProvider,  
    yahooProvider,  
    secondaryProvider  
];  
  
for (const provider of providers) {  
    try {  
        const result =  
            await  
provider.getQuote(symbol);  
  
        if (validateQuote(result))  
{  
            return result;  
        }  
  
    } catch (error) {  
  
        logProviderFailure(  
            provider.name,  
            symbol,  
            error  
        )  
    }  
}
```

```
) ;
```

```
}
```

```
}
```

```
throw new  
MarketDataUnavailableError(symbol);  
}
```

Only after all configured providers fail should the AI report that live data is unavailable.

15. NORMALIZED DATA OBJECT

Every market data result should include:

```
{  
  "symbol": "SPY",  
  "price": 0,  
  "previous_close": 0,  
  "change": 0,  
  "change_percent": 0,  
  "volume": 0,  
  "market_session": "REGULAR",
```

```
“source”: “provider_name”,  
“timestamp”: “ISO-8601”,  
“freshness_seconds”: 0,  
“verified”: true  
}
```

Never pass an unlabeled price to the LLM.

16. DATA FRESHNESS GATE

Before displaying current information:

```
function isFresh(data, maximumAgeSeconds) {  
  
  const age =  
    (Date.now() -  
     new  
Date(data.timestamp).getTime()) /  
1000;  
  
  return age <= maximumAgeSeconds;  
}
```

Suggested starting parameters:

REGULAR MARKET:
60 seconds

PREMARKET:

300 seconds

AFTER HOURS:

300 seconds

MARKET CLOSED:

Use final verified session values and explicitly label them as the latest completed session.

Do not represent stale cached data as live.

17. MARKET SNAPSHOT CACHE

Build a centralized:

MarketSnapshotService

Instead of calculating the entire U.S. market separately for every conversation.

Example object:

```
{  
  "timestamp": "...",  
  "session": "REGULAR",  
  "market_health": 72,  
  "regime": "BULL",  
  "indexes": {...},
```

```
“breadth”: {...},  
“sectors”: [...],  
“volatility”: {...},  
“macro_events”: [...],  
“major_news”: [...]  
}
```

Refresh automatically according to market session.

Then:

“How’s the market?”

can retrieve the latest verified MarketSnapshot.

18. EARNINGS CALENDAR REVISION

The earnings calendar needs to become deterministic.

If intent = EARNINGS_CALENDAR:

ALWAYS execute:

getEarningsCalendar()

Do not leave this decision to the LLM.

Normalize results into:

```
{  
  "ticker": "",  
  "company": "",  
  "report_date": "",  
  "report_time": "AMC | BMO | UNKNOWN",  
  
  "eps_estimate": null,  
  "revenue_estimate": null,  
  
  "market_cap": null,  
  
  "sector": "",  
  "industry": "",  
  
  "date_status":  
  "CONFIRMED | ESTIMATED",  
  
  "source": "",  
  
  "last_updated": ""  
}
```

19. CALENDAR UI RENDERING

The UI should render the calendar based on backend state.

Example:

```
if (  
  intent === "EARNINGS_CALENDAR" &&  
  earnings.length > 0  
) {
```

```
renderEarningsCalendar(earnings);  
}
```

Do NOT depend on the LLM to decide whether the calendar should appear.

The LLM provides the explanation.

The application controls the UI.

20. GENERAL CALENDAR ENGINE

Eventually use the same calendar architecture for:

Quarterly Earnings

Economic Events

CPI

PPI

FOMC

Federal Reserve announcements

Jobs Report

GDP

Options expiration

Stock splits

Dividends

Major corporate events

Each event should have:

event_type
date
time
importance
affected_symbols
affected_sectors
source
verified
timestamp

21. PREDETERMINED LLM CONTEXT

Once calculations are complete, construct the LLM context.

Example:

```
const marketContext = {  
  
  intent,  
  
  market: "US",  
  
  session,  
  
  timestamp,
```

marketHealth,

regime,

indexes,

breadth,

sectors,

volatility,

macroEvents,

news,

candidates

};

The LLM receives this object together with the user's question.

22. LLM SYSTEM INSTRUCTION

Add a permanent instruction similar to:

“You are Requi Trading Intelligence.

For current market questions, never generate market prices, market conditions, earnings dates, technical indicators, market movers, or stock recommendations from model memory.

Use only structured market data supplied by Requi's verified backend services.

When the user does not specify a market, assume U.S. equities traded on NYSE/NASDAQ.

Do not require a ticker for general market questions.

For beginner questions, interpret the user's intent and explain the supplied market intelligence in simple conversational language.

Never invent missing numerical data.

If one field is unavailable, continue the analysis using available verified fields when sufficient rather than refusing the entire request.

Clearly distinguish live, delayed, cached, estimated, and unavailable information.

When stock candidates are supplied by the Stock Discovery Engine, explain their ranking, catalysts, market context, and risk. Do not independently create additional ticker recommendations.”

23. PARTIAL DATA FAILURE

This is important.

One missing data point should NOT cause the entire response to fail.

Example:

SPY = AVAILABLE

QQQ = AVAILABLE

DIA = AVAILABLE

IWM = AVAILABLE

VIX = AVAILABLE

Breadth = AVAILABLE

Sector Data = AVAILABLE

One macro feed = FAILED

The system should still answer.

Return:

analysis_confidence = 0.91

```
missing_fields = [  
  "economic_calendar"  
]
```

The LLM can say:

“The U.S. market is broadly positive based on index, breadth, sector and volatility data. Today’s economic-calendar feed is temporarily unavailable.”

This is much better than:

“I don’t have verified market data.”

24. ANALYSIS CONFIDENCE SCORE

Add a confidence score based on data completeness and freshness.

Example:

$\text{DATA_COMPLETENESS} = \frac{\text{available_required_fields}}{\text{total_required_fields}}$

$\text{FRESHNESS_SCORE} = \text{weighted freshness of datasets}$

$\text{SOURCE_CONFIDENCE} = \text{provider reliability score}$

$$\begin{aligned} \text{ANALYSIS_CONFIDENCE} = & \\ & (\text{DATA_COMPLETENESS} \times 0.50) \\ & + \\ & (\text{FRESHNESS_SCORE} \times 0.30) \\ & + \\ & (\text{SOURCE_CONFIDENCE} \times 0.20) \end{aligned}$$

Possible classifications:

90–100%

HIGH CONFIDENCE

75–89%

GOOD CONFIDENCE

60–74%

MODERATE CONFIDENCE

<60%

LIMITED DATA

The confidence score measures the quality/completeness of the analysis inputs. It must NOT be presented as a probability that the market will rise or that a stock recommendation will succeed.

25. OBSERVABILITY / DEBUGGING

Every intelligence request should generate an internal trace.

Example:

request_id

user_query

detected_intent

market

market_session

functions_called

providers_called

provider_failures

cache_hit

calculation_version

market_health_score

market_regime

analysis_confidence

response_latency

LLM_model

This is critical because when Requi gives a bad answer we need to determine whether the problem came from:

Intent classification

Data retrieval

Provider failure

Calculation

Cache

LLM interpretation

or UI rendering.

26. VERSION THE CALCULATIONS

Do not bury market formulas inside prompts.

Create versions:

MARKET_HEALTH_V1
REGIME_MODEL_V1
STOCK_RANKING_V1
SECTOR_MODEL_V1
BREADTH_MODEL_V1

Store:

model_version
calculation_timestamp
inputs
output

This allows us to modify formulas later and compare performance.

27. RESPONSE BEHAVIOR

The final response should remain conversational.

Do not expose all of the technical calculations unless the user requests them.

For example:

USER:

“How’s the market today?”

REQUI:

“The U.S. market looks moderately bullish today.

The major indexes are mostly higher, market breadth is positive, and technology is currently showing the strongest relative strength. Volatility remains within a normal range.

Market Health: 72/100

Regime: Bullish

Technology and semiconductors are showing the strongest leadership, while defensive sectors are lagging.

There are several stocks showing strong relative strength and above-average volume. I can show you the strongest opportunities we’re detecting today.”

Behind this simple response should be the entire deterministic analysis pipeline.

28. “WHAT SHOULD I BUY TODAY?” RESPONSE FLOW

USER:

“What stock should I buy today?”

SYSTEM:

1. Assume US market.
 2. Determine current session.
 3. Retrieve MarketSnapshot.
 4. Calculate market regime.
 5. Identify leading sectors.
 6. Identify leading industries.
 7. Run stock universe screen.
 8. Calculate candidate scores.
 9. Validate prices/data.
 10. Check relevant news/catalysts.
 11. Apply risk filters.
 12. Rank candidates.
 13. Send structured candidates to LLM.
 14. Generate conversational explanation.
- The LLM should never select a stock solely because it “knows” the company.

29. EARNINGS QUESTION FLOW

USER:

“Who’s reporting earnings tomorrow?”

SYSTEM:

1. intent = EARNINGS_CALEDNDAR
2. market = US
3. Resolve tomorrow using America/New_York.
4. Query earnings service.
5. Run fallback if necessary.
6. Validate events.
7. Separate confirmed vs estimated.
8. Classify AMC/BMO.
9. Render calendar.
10. Send structured events to LLM.
11. LLM summarizes notable reports.

30. DESIRED PRODUCT BEHAVIOR

Requi should be designed around:

BEGINNER INPUT

→ PROFESSIONAL INTERPRETATION

→ VERIFIED DATA

→ DETERMINISTIC ANALYSIS

→ SIMPLE RESPONSE

The intelligence should live behind the interface.

Users should NOT need to understand:

Tickers

Market breadth

Relative strength

VIX

Sector rotation

RVOL

SMA

Market regimes

Economic calendars

or earnings terminology.

They should simply be able to ask:

“How’s the market?”

“What’s good today?”

“What should I watch?”

“What’s going down?”

“Who has earnings?”

“Is it risky today?”

and Requi should automatically execute the appropriate professional analysis.

31. IMPORTANT DEVELOPMENT PRINCIPLE

The architecture should follow:

LLM = LANGUAGE + INTENT + EXPLANATION

BACKEND = DATA + MATH + VALIDATION +
RANKING

APPLICATION = ORCHESTRATION + UI

The LLM should not be the market-data engine.

The LLM should be the intelligent interface sitting on top
of the market-data engine.

32. IMPLEMENTATION PRIORITY

PHASE 1

Default U.S. market behavior

Intent Router

General Market Snapshot

Index analysis

Market Health Score

Market Regime

Sector rankings

VIX/volatility

Yahoo/broker data fallback

Market Snapshot caching

PHASE 2

Stock Discovery Engine

Relative strength

RVOL

Momentum ranking

Industry ranking

Catalyst/news integration

Candidate scoring

PHASE 3

Earnings Calendar reliability

Confirmed vs estimated earnings

AMC/BMO classification

Automatic calendar rendering

Economic calendar integration

PHASE 4

Confidence scoring

Full observability

Calculation versioning

Historical storage

Backtesting and calibration

FINAL REQUIREMENT

The key change is that Requi should no longer wait for the LLM to decide how to answer general market questions.

The application should already know:

IF GENERAL MARKET QUESTION

→ RUN MARKET INTELLIGENCE

IF STOCK DISCOVERY QUESTION

→ RUN MARKET + SECTOR + STOCK DISCOVERY

IF EARNINGS QUESTION

→ RUN EARNINGS ENGINE + DISPLAY

CALENDAR

IF TICKER QUESTION

→ RUN TICKER INTELLIGENCE

IF MARKET MOVERS QUESTION

→ RUN MOVERS + VOLUME + CATALYST
ANALYSIS

The LLM receives the results and communicates them naturally.

The goal is not simply to make the LLM smarter.

The goal is to make Requi's application architecture intelligent enough that the correct analysis is automatically performed before the LLM responds.

That should make the product significantly more reliable, beginner-friendly, testable, scalable, and resistant to hallucination.